Problem: The "RUET-Archive" Optimal Compressor

The RUET Central Library is digitizing old manuscripts. To save storage space, they need a custom compression tool. The text in these manuscripts uses a small set of characters with very high repetition.

You are tasked with building the "RUET-Archive" engine. Your goal is to use a **greedy approach** to build an optimal, **prefix-free binary coding** scheme. A prefix-free code ensures that no character's code is the prefix of another's, which allows for unambiguous decoding.
Given Data:

An analysis of a sample manuscript page yielded the following character frequencies:

Character	'a'	'd'	'e'	'n'	'o'	'r'	's'	't'	'_'
Frequency	10	8	25	14	7	12	18	6	3

Sample Text to Encode: send_data_to_node
(Note: The character '_' (underscore) is used in the sample text. Its frequency is 3.)

Your tasks are as follows:

1. (2 Marks) Data Structure Setup:

- Define a Node structure (or class) for the binary tree. It must store the character, its frequency, and pointers to its left and right children.
- Define a custom comparator (or overload the < operator) for your Node structure so it can be correctly used in a **min-priority queue**. The queue must prioritize nodes with the **lowest frequency**.

2. (6 Marks) Optimal Tree Construction:

- Implement the core **greedy algorithm** to build the optimal tree:
  1. Create a leaf node for each character (including the '_' character, freq: 3) and add them all to a min-priority queue.
  2. While more than one node remains in the queue:
     - Extract the two nodes with the minimum frequencies (let's call them left and right).
     - Create a new internal parent node. Its frequency should be the **sum** of left's and right's frequencies.
     - Set left and right as the children of this new parent node.
     - Insert the new parent node back into the priority queue.
- The final node remaining in the queue is the root of your optimal tree.

3. (5 Marks) Code Generation:

- Implement a function (e.g., a recursive traversal) that starts from the tree's root.
- This function should generate the binary code for each character. (Convention: '0' for left, '1' for right).
- **Output 1:** Print a table clearly showing each character, its frequency, and its final binary code.

4. (4 Marks) Encoding and Analysis:

- **Output 2 (Encoding):** Using your generated codes, encode the sample text: send_data_to_node. Print the resulting single binary string.

- **Output 3 (Analysis):** Calculate and print the following:
  - **A)** The total number of bits required for the encoded string (from Output 2).
  - **B)** The number of bits that would have been required using a **fixed-length code**. (There are 9 unique characters in the frequency table).

5. (3 Marks) Decoding:

- Implement a decode function that takes the root of your tree and the following binary string:

  101110011010100001110001001000111 1

- Your function should traverse the tree from the root, following the bits ('0' = left, '1' = right) until it reaches a leaf node. It should then append that leaf's character to the result and restart the traversal from the root for the next bit.
- **Output 4 (Decoding):** Print the original text decoded from the bitstring.

Implementation Guidelines:

- You **must** use a built-in **min-priority queue** (e.g., priority_queue in C++, PriorityQueue in Java, heapq in Python) as the core of your greedy algorithm.
- Your code should be clean and well-commented.
- **Tie-breaking:** If two nodes have the same frequency, the priority queue's default behavior is acceptable.
- Store your generated codes in a map or dictionary for fast lookup during the encoding phase.